

Guide de Survie Ultime — Google Cloud Platform (GCP)

Version 2026 | Le document a garder sous le coude (ou dans un depot Git prive) pour les vendredis apres-midi a 16h55.

Table des Matieres

-  Posture de Crise : La Methode B.E.S.T.
-  IAM & Securite : La Premiere Ligne de Defense
-  Kit de Secours Python (Scripts Reutilisables)
-  One-Liners Cloud Shell (Bash / CLI)
-  Cloud Storage : Le Bucket Instantane
-  Reseau & Securite Forteresse
-  Compute Engine : VM Express & Docker
-  Kubernetes (GKE) : Le Kit de Reanimation
-  Docker Engine : Le Grand Nettoyage
-  Cloud SQL : Sauvetage de Base de Donnees
-  Monitoring, Logging & Alerting
-  Antiseche : Ca ne marche pas, je fais quoi ?
-  Checklist Vendredi Soir
-  Nettoyage Post-Crise

Posture de Crise : La Methode B.E.S.T.

Avant de lancer le moindre script, **respire un coup** et applique la methode **B.E.S.T.** :

Lettre	Action	Pourquoi
B ackup	Snapshot / export de la config actuelle	On ne touche a rien sans filet
E xplain	Ecrire ce qu on va faire <i>avant</i> de le faire	Evite les oops irreversibles
S imulate	Utiliser <code>--dry-run</code> ou l equivalent Python	Tester sans casser
T rack	Logger chaque commande et son retour	Reproductibilite & audit

 **Astuce Pro** : Cree un canal Slack prive `#war-room-[date]` et colle chaque commande executee avec son timestamp. C est ton journal de bord numerique.

IAM & Securite : La Premiere Ligne de Defense

1. Verifier les permissions critiques en urgence

```
# Lister les membres avec des roles d admin
for role in $(gcloud iam roles list --format="value(name)" | grep -i admin); do
```

```

echo "=== $role ==="
gcloud projects get-iam-policy $(gcloud config get-value project) \
  --flatten="bindings[].members" \
  --format="table(bindings.role,bindings.members)" \
  --filter="bindings.role=$role"
done

```

2. Activer MFA sur tous les comptes (obligatoire)

```

# Verifier les comptes sans MFA
gcloud identity groups memberships list --group-email="admins@company.com"
↪ --format="table(memberKey.id,role)"

```

3. Rotation d'urgence d'une clé de service account

```

# Lister les clés gérées par l'utilisateur (à éviter !)
gcloud iam service-accounts keys list \
  --iam-account=SA_NAME@PROJECT_ID.iam.gserviceaccount.com \
  --managed-by=user

# Supprimer une clé compromise
gcloud iam service-accounts keys delete KEY_ID \
  --iam-account=SA_NAME@PROJECT_ID.iam.gserviceaccount.com

# Créer une nouvelle clé (temporaire, à supprimer après usage)
gcloud iam service-accounts keys create new-key.json \
  --iam-account=SA_NAME@PROJECT_ID.iam.gserviceaccount.com

```

4. Bloquer l'accès public sur TOUS les buckets

```

# Lister les buckets potentiellement publics
gsutil ls | while read bucket; do
  gsutil iam get "$bucket" | grep -E "allUsers|allAuthenticatedUsers" && echo
  ↪ "$${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/doc/miner/emoji/
  ↪ 26a0-fe0f.png}}}}}}}$${ PUBLIC: $bucket"
done

# Corriger immédiatement
gsutil iam ch -d allUsers gs://BUCKET_NAME
gsutil iam ch -d allAuthenticatedUsers gs://BUCKET_NAME

```

Kit de Secours Python (Scripts Réutilisables)

1. Le Panic Button : Rotation d'urgence de secret (Secret Manager)

```

import os
from google.cloud import secretmanager

def rotate_secret_gcp(project_id, secret_id, new_value):

```

```

""" Met a jour un secret GCP et force la creation d une nouvelle version.
"""
client = secretmanager.SecretManagerServiceClient()
parent = f"projects/{project_id}/secrets/{secret_id}"

try:
    print(f"[INFO] Initialisation de la rotation pour : {secret_id}")

    response = client.add_secret_version(
        request={"parent": parent, "payload": {"data": new_value.encode("UTF-8")}}
    )
    print(f"[SUCCESS] Nouvelle version pousse. Name: {response.name}")

    client.update_secret(
        request={
            "secret": {"name": parent, "replication": {"automatic": {}}},
            "update_mask": {"paths": ["replication"]},
        }
    )

except Exception as e:
    print(f"[ERROR] Echec de la rotation : {str(e)}")
    raise

# Utilisation rapide en Cloud Shell :
# rotate_secret_gcp("mon-project-id", "prod/api/stripe", "sk_live_51Nx...")

```

2. Redimensionnement d Instance avec Rollback Automatique

```

import time
import subprocess
import json

def resize_with_rollback_gcp(instance_name, zone, new_machine_type, old_machine_type):
    """ Modifie le type d une VM GCP. Rollback auto si health check echoue.
    """
    print(f"[1/5] Arret de l instance {instance_name}...")
    subprocess.run([
        "gcloud", "compute", "instances", "stop", instance_name,
        "--zone", zone, "--quiet"
    ], check=True)

    print(f"[2/5] Modification vers {new_machine_type}...")
    subprocess.run([
        "gcloud", "compute", "instances", "set-machine-type", instance_name,
        "--zone", zone, "--machine-type", new_machine_type
    ], check=True)

    print(f"[3/5] Redemarrage...")
    subprocess.run([
        "gcloud", "compute", "instances", "start", instance_name,
        "--zone", zone
    ], check=True)

```

```

print("[4/5] Attente du Health Check (5 minutes)...")
time.sleep(300)

result = subprocess.run([
    "gcloud", "compute", "instances", "describe", instance_name,
    "--zone", zone, "--format=json"
], capture_output=True, text=True)

data = json.loads(result.stdout)
status = data.get("status", "UNKNOWN")
health_ok = status == "RUNNING"

if not health_ok:
    print("[${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/26a0-fe0f.png}}}}] CRITIQUE] Health Check echoue ! Lancement du ROLLBACK...")
    subprocess.run([
        "gcloud", "compute", "instances", "stop", instance_name,
        "--zone", zone, "--quiet"
    ], check=True)
    subprocess.run([
        "gcloud", "compute", "instances", "set-machine-type", instance_name,
        "--zone", zone, "--machine-type", old_machine_type
    ], check=True)
    subprocess.run([
        "gcloud", "compute", "instances", "start", instance_name,
        "--zone", zone
    ], check=True)
    print("[${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/21a9-fe0f.png}}}}] ROLLBACK EFFECTUE] Instance remise dans son etat d origine.")
else:
    print("[${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/1f389.png}}}}] SUCCES] L instance est stable. Bon week-end !")

# Exemple d appel :
# resize_with_rollback_gcp("vm-prod-01", "europe-west1-b", "e2-standard-4", "e2-medium")

```

3. Deploiement d une regle de pare-feu temporaire (IP dynamique)

```

import urllib.request
import json
import os
import time

def deploy_emergency_firewall(project_id, network_name="default"):
    """
    Recupere l IP publique actuelle et cree une regle de pare-feu temporaire.
    """
    with urllib.request.urlopen("https://api.ipify.org?format=json") as response:
        my_ip = json.loads(response.read().decode())['ip']
        print(f"[INFO] IP publique detectee : {my_ip}")

    rule_name = f"allow-admin-temp-{int(time.time())}"

```

```

cmd = (
    f"gcloud compute firewall-rules create {rule_name} "
    f"--project={project_id} "
    f"--network={network_name} "
    f"--allow=tcp:22,tcp:443 "
    f"--source-ranges={my_ip}/32 "
    f"--description='Acces urgence - {time.strftime(\"%Y-%m-%d %H:%M\")}'"
)

print(f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
↪ 26a1.png}}}}}}$ ACTION] Deploiement de la regle : {rule_name}...")
os.system(cmd)
print(f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
↪ 2705.png}}}}}}$ Regle {rule_name} active. A supprimer lundi matin !")
return rule_name

# Utilisation : deploy_emergency_firewall("mon-project-id")

```

📁 One-Liners Cloud Shell (Bash / CLI)

🔍 Audit & Forensics

Qui a modifié QUOI au cours des 2 dernières heures ?

```

# GCP Cloud Logging - Audit des actions admin
gcloud logging read "protoPayload.authenticationInfo.principalEmail!=" AND timestamp>="$(date -u -d
↪ '2 hours ago' +%Y-%m-%dT%H:%M:%SZ)\"" \
--format="table(timestamp, protoPayload.authenticationInfo.principalEmail, protoPayload.methodName,
↪ protoPayload.resourceLabels)"

```

Lister toutes les ressources créées aujourd'hui

```

gcloud asset search-all-resources \
--query="createTime>=$(date -u +%Y-%m-%dT00:00:00Z)" \
--format="table(displayName, assetType, createTime, project)"

```

Vérifier les buckets publics en une ligne

```

gsutil ls | xargs -I {} gsutil iam get {} | grep -B2 -A2 "allUsers\|allAuthenticatedUsers"

```

📁 Snapshots & Backups

Snapshot de sécurité instantané avant manipulation

```

# GCP Compute Engine - Snapshot disque
gcloud compute snapshots create snap-urgence-$(date +%s) \
--source-disk=mon-disque-prod \
--source-disk-zone=europe-west1-b \
--description="Snapshot urgence avant maintenance $(date)"

```

```
# Verifier le snapshot
gcloud compute snapshots list --filter="name~'snap-urgence'" --format="table(name, status, diskSizeGb,
↳ creationTimestamp)"
```

Backup automatique d'une base Cloud SQL

```
gcloud sql backups create --instance=ma-base-prod --description="Backup urgence $(date +%Y%m%d-%H%M)"
```

Reseau & Connectivite

Tester la connectivite reseau entre 2 VMs

```
gcloud compute networks subnets list --format="table(name, region, ipCidrRange)"
# Puis depuis la VM source :
curl -s -o /dev/null -w "%{http_code}" http://IP_DESTINATION:PORT
```

Verifier les regles de pare-feu actives

```
gcloud compute firewall-rules list \
--format="table(name, network, direction, allowed, sourceRanges, disabled)" \
--sort-by=network
```

Cloud Storage : Le Bucket Instantane

Creation ultra-securisee d'un bucket de secours

```
# Nom unique (obligatoire)
BUCKET_NAME="mon-bucket-secours-$(date +%s)"

# Creation du bucket HA avec verrouillage public
gcloud storage buckets create gs://${BUCKET_NAME} \
--location=europe-west1 \
--uniform-bucket-level-access

# Forcer l'interdiction de l'accès public
gcloud storage buckets update gs://${BUCKET_NAME} \
--update-public-access-prevention=enforced

# Activer le versioning (protection contre suppression accidentelle)
gcloud storage buckets update gs://${BUCKET_NAME} \
--versioning

# Activer le soft delete (recuperation 7 jours apres suppression)
gcloud storage buckets update gs://${BUCKET_NAME} \
--soft-delete-duration=7d

echo "$${vcenter}{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
↳ 2705.png}}}}}$ Bucket cree : gs://${BUCKET_NAME}"
```

Upload rapide avec verification d integrite

```
# Upload avec checksum MD5
gsutil cp -h "Content-MD5:${(openssl md5 -binary fichier.zip | base64)}" fichier.zip gs://${BUCKET_NAME}/

# Verifier
gsutil ls -L gs://${BUCKET_NAME}/fichier.zip | grep "Hash (md5)"
```

Reseau & Securite Forteresse

1. Creer un VPC isole avec sous-reseaux segmentes

```
# Creer le VPC
gcloud compute networks create vpc-crise \
  --subnet-mode=custom \
  --bgp-routing-mode=regional

# Sous-reseau Production
gcloud compute networks subnets create subnet-prod \
  --network=vpc-crise \
  --region=europe-west1 \
  --range=10.0.1.0/24 \
  --enable-private-ip-google-access

# Sous-reseau Bastion (acces admin)
gcloud compute networks subnets create subnet-bastion \
  --network=vpc-crise \
  --region=europe-west1 \
  --range=10.0.255.0/24

# Activer les VPC Flow Logs (forensics)
gcloud compute networks subnets update subnet-prod \
  --region=europe-west1 \
  --enable-flow-logs
```

2. Regles de pare-feu minimalistes

```
# Autoriser uniquement le bastion a SSH sur la prod
gcloud compute firewall-rules create allow-ssh-from-bastion \
  --network=vpc-crise \
  --allow=tcp:22 \
  --source-ranges=10.0.255.0/24 \
  --target-tags=prod-vm

# Autoriser HTTPS depuis Cloud Load Balancer uniquement
gcloud compute firewall-rules create allow-https-from-lb \
  --network=vpc-crise \
  --allow=tcp:443 \
  --source-ranges=130.211.0.0/22,35.191.0.0/16 \
  --target-tags=prod-vm
```

```
# Bloquer tout le reste (regle implicite de refus, mais explicite ici)
gcloud compute firewall-rules create deny-all-ingress \
  --network=vpc-crise \
  --action=DENY \
  --rules=all \
  --source-ranges=0.0.0.0/0 \
  --priority=1000
```

3. Cloud NAT pour les instances privées

```
# Creer une IP statique pour le NAT
gcloud compute addresses create nat-ip-1 \
  --region=europe-west1

# Creer le routeur Cloud NAT
gcloud compute routers create router-nat \
  --network=vpc-crise \
  --region=europe-west1

gcloud compute routers nats create nat-config \
  --router=router-nat \
  --region=europe-west1 \
  --nat-all-subnet-ip-ranges \
  --nat-external-ip-pool=nat-ip-1
```

■ Compute Engine : VM Express & Docker

1. Deploiement d une VM Container-Optimized (COS)

```
gcloud compute instances create-with-container vm-secours-docker \
  --zone=europe-west1-b \
  --machine-type=e2-medium \
  --container-image=gcr.io/cloud-marketplaces/google/nginx1:latest \
  --container-restart-policy=always \
  --tags=http-server,https-server \
  --metadata=startup-script="#!/bin/bash
echo 'VM lancee en speed le $(date)' > /var/log/emergency.log" \
  --shielded-secure-boot \
  --shielded-integrity-monitoring
```

2. Lancer un Docker en urgence sur une VM existante via SSH

```
import subprocess
import sys

def run_emergency_docker(instance_name, zone, docker_image, env_vars="", port_mapping="80:80"):
    """
    Deploie un conteneur Docker de secours sur une VM GCP existante.
    """
    print(f"[1/3] Verification de l instance {instance_name}...")
```



```

check = subprocess.run(
    ["gcloud", "compute", "instances", "describe", instance_name, "--zone", zone,
     ↪ "--format=value(status)"],
    capture_output=True, text=True
)
if check.returncode != 0 or "RUNNING" not in check.stdout:
    print(f"[$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
     ↪ 274c.png}}}}}}]$] Instance {instance_name} non accessible.")
    sys.exit(1)

print(f"[2/3] Nettoyage des anciens conteneurs...")
cleanup_cmd = (
    f"gcloud compute ssh {instance_name} --zone={zone} "
    f"--command='sudo docker stop app-prod 2>/dev/null; sudo docker rm app-prod 2>/dev/null; true'"
)
subprocess.run(cleanup_cmd, shell=True)

print(f"[3/3] Lancement du conteneur {docker_image}...")
run_cmd = (
    f"gcloud compute ssh {instance_name} --zone={zone} "
    f"--command='sudo docker run -d --name app-prod --restart always -p {port_mapping} {env_vars}
     ↪ {docker_image}'"
)
result = subprocess.run(run_cmd, shell=True, capture_output=True, text=True)

if result.returncode == 0:
    print(f"[$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
     ↪ 1f389.png}}}}}}]$] SUCCES] Conteneur en ligne avec restart automatique.")
    verify = subprocess.run(
        f"gcloud compute ssh {instance_name} --zone={zone} --command='sudo docker ps --filter
         ↪ name=app-prod'",
        shell=True, capture_output=True, text=True
    )
    print(verify.stdout)
else:
    print(f"[$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
     ↪ 26a0-fe0f.png}}}}}}]$] ERREUR] Echec : {result.stderr}")

# Exemple d appel :
# run_emergency_docker("vm-prod-01", "europe-west1-b", "nginx:alpine", "-e ENV=prod", "8080:80")

```

3. Recuperer l IP interne d une VM (pour config K8s / reverse proxy)

```

# Via gcloud CLI
gcloud compute instances describe vm-interne \
    --zone=europe-west1-b \
    --format="value(networkInterfaces[0].networkIP)"

```

```

import subprocess
import json

```

```
def get_vm_internal_ip(instance_name, zone):
    """    Recupere 1 IP privée d une VM GCP.
    """
    cmd = f"gcloud compute instances describe {instance_name} --zone={zone} --format=json"
    res = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    if res.returncode == 0:
        data = json.loads(res.stdout)
        return data['networkInterfaces'][0]['networkIP']
    return None

# print(get_vm_internal_ip("k8s-node-01", "europe-west1-b"))
```

Kubernetes (GKE) : Le Kit de Reanimation

1. One-Liner de Triage Global

```
# Lister tous les pods hors-service dans tous les namespaces
kubectl get pods --all-namespaces | grep -E -v 'Running|Completed'

# Version plus detaillee avec raison du crash
kubectl get pods --all-namespaces -o json | jq -r '.items[] | select(.status.phase != "Running" and
↳ .status.phase != "Succeeded") | "\(.metadata.namespace)/\(.metadata.name): \(.status.phase) -
↳ \(.status.containerStatuses[0].state.waiting.reason // "N/A")"'
```

2. Redemarrage Zero Downtime (Rolling Restart)

```
# Forcer un rolling restart d un deploiement
kubectl rollout restart deployment/mon-app-critique -n production

# Surveiller le rollout en temps reel
kubectl rollout status deployment/mon-app-critique -n production --watch

# En cas d echec, rollback immediat
kubectl rollout undo deployment/mon-app-critique -n production
```

3. Script Python : Le Crash Watcher de Secours

```
import time
from kubernetes import client, config
from datetime import datetime

def watch_deployment_health(namespace, deployment_name, alert_threshold=3):
    """    Surveille les pods d un deploiement et alerte en cas d erreur.
    """
    try:
        config.load_kube_config()
    except Exception:
        print("[${\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
↳ 274c.png}}}}] Impossible de charger kubeconfig. Verifiez 'gcloud container clusters
↳ get-credentials'")
    return
```

```

v1 = client.CoreV1Api()
print(f"[$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f440.png}}}}}}$$] Surveillance du namespace '{namespace}' lancee (Ctrl+C pour arreter)...")
print(f"[INFO] Seuil d alerte : {alert_threshold} crashes consecutifs")

try:
    while True:
        pods = v1.list_namespaced_pod(
            namespace,
            label_selector=f"app={deployment_name}"
        )

        current_crash = 0
        for pod in pods.items:
            pod_name = pod.metadata.name
            if pod.status.container_statuses:
                for container in pod.status.container_statuses:
                    if container.state.waiting:
                        reason = container.state.waiting.reason
                        if reason in ["CrashLoopBackOff", "ErrImagePull", "ImagePullBackOff"]:
                            current_crash += 1
                            print(f"[$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/26a0-fe0f.png}}}}}}$$ ALERTE]
                                ↳ {datetime.now().isoformat()} | Pod {pod_name} : {reason}")
                            events = v1.list_namespaced_event(
                                namespace,
                                field_selector=f"involvedObject.name={pod_name}"
                            )
                            for event in events.items[:3]:
                                print(f"    ↳ Event: {event.reason} - {event.message}")

            if current_crash >= alert_threshold:
                print(f"[$$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/1f6a8.png}}}}}}$$ CRITIQUE] {current_crash} pods en erreur ! Action de
                    ↳ rollback recommandee.")

                time.sleep(10)

        except KeyboardInterrupt:
            print("
[👋] Surveillance arretee.")

# watch_deployment_health("prod", "api-gateway")

```

4. GKE : Acces securise au master

```

# Restreindre l acces au master GKE a des IPs specifiques
gcloud container clusters update CLUSTER_NAME \
    --zone=europe-west1-b \
    --enable-master-authorized-networks \

```

```
--master-authorized-networks=$(curl -s https://api.ipify.org)/32

# Verifier
gcloud container clusters describe CLUSTER_NAME --zone=europe-west1-b
↪ --format="value(masterAuthorizedNetworksConfig)"
```

Docker Engine : Le Grand Nettoyage

La purge d urgence (Disque plein a 99%)

```
# Nettoyer TOUS les conteneurs arretes, reseaux inutilises, images orphelines
docker system prune -a --volumes -f

# Si vraiment critique : supprimer aussi les images non utilisees
docker image prune -a -f

# Verifier l espace libere
docker system df
```

Extraire les logs d un conteneur en boucle

```
# Les 100 dernieres lignes + suivi en temps reel
docker logs --tail 100 -f nom_du_conteneur_en_panique

# Logs avec timestamp
docker logs --tail 100 -f --timestamps nom_du_conteneur_en_panique

# Rediriger vers un fichier pour analyse
docker logs --tail 500 nom_du_conteneur_en_panique > /tmp/crash_logs_$(date +%s).txt 2>&1
```

Diagnostic rapide d un conteneur bloque

```
# Entrer dans le conteneur en urgence
docker exec -it nom_du_conteneur /bin/sh

# Ou si le conteneur est en crash loop
docker run --rm -it --entrypoint /bin/sh nom_image:latest

# Verifier les ressources consommees
docker stats --no-stream --format "table {{.Name}} {{.CPUPerc}} {{.MemUsage}} {{.NetIO}}"
```

Cloud SQL : Sauvetage de Base de Donnees

1. Forcer SSL sur une instance

```
gcloud sql instances patch INSTANCE_NAME --require-ssl

# Verifier
```

```
gcloud sql instances describe INSTANCE_NAME --format="value(settings.ipConfiguration.requireSsl)"
```

2. Restreindre les IPs autorisees

```
# Remplacer les IPs autorisees (supprime les anciennes !)  
gcloud sql instances patch INSTANCE_NAME \  
  --authorized-networks=IP_BUREAU/32,IP_VPN/32  
  
# Jamais 0.0.0.0/0 en production !
```

3. Backup & Restore en urgence

```
# Creer un backup manuel  
gcloud sql backups create --instance=INSTANCE_NAME --description="Urgence $(date +%Y%m%d-%H%M)"  
  
# Lister les backups disponibles  
gcloud sql backups list --instance=INSTANCE_NAME  
  
# Restaurer depuis un backup (ATTENTION : ecrase les donnees actuelles !)  
gcloud sql instances restore INSTANCE_NAME --backup-id=BACKUP_ID  
  
# Alternative : creer une instance clone pour investigation  
gcloud sql instances clone INSTANCE_NAME INSTANCE_NAME-clone-$(date +%s)
```

4. Connexion via Cloud SQL Proxy (securise, sans IP publique)

```
# Telecharger le proxy (si pas deja fait)  
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.8.0/  
↪ cloud-sql-proxy.linux.amd64  
chmod +x cloud-sql-proxy  
  
# Connexion a une instance privree via IAM  
./cloud-sql-proxy --port 5432 PROJECT:REGION:INSTANCE_NAME  
  
# Dans un autre terminal :  
psql -h 127.0.0.1 -p 5432 -U USER -d DATABASE
```

Monitoring, Logging & Alerting

1. Creer une alerte d urgence via Cloud Monitoring

```
# Creer une alerte sur CPU > 90% pendant 5 minutes  
gcloud alpha monitoring policies create \  
  --display-name="CPU Critique - VM Prod" \  
  --condition-display-name="CPU > 90%" \  
  --condition-filter="metric.type=\"compute.googleapis.com/instance/cpu/utilization\" AND  
  ↪ resource.type=\"gce_instance\" AND metric.labels.instance_name=\"vm-prod-01\"" \  
  --condition-comparison=COMPARISON_GT \  
  --condition-threshold-value=0.9 \  
  --condition-duration=5m
```

```
--condition-threshold-duration=300s \
--notification-channel="projects/PROJECT/notificationChannels/CHANNEL_ID"
```

2. Requetes Log Analytics rapides

```
# Rechercher les erreurs 500 sur un Load Balancer
gcloud logging read "resource.type=http_load_balancer AND httpRequest.status>=500 AND
↳ timestamp>=\"$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)\" \" \
--format="table(timestamp, jsonPayload.statusDetails, httpRequest.requestUrl, httpRequest.remoteIp)"

# Rechercher les echecs d authentication IAM
gcloud logging read "protoPayload.methodName="google.login" AND protoPayload.status.message!="OK" AND
↳ timestamp>=\"$(date -u -d '24 hours ago' +%Y-%m-%dT%H:%M:%SZ)\" \" \
--format="table(timestamp, protoPayload.authenticationInfo.principalEmail,
↳ protoPayload.status.message)"
```

3. Dashboard d urgence Cloud Monitoring

```
# Creer un dashboard de crise
gcloud monitoring dashboards create \
--dashboard-json='{
  "displayName": "Dashboard Crise",
  "gridLayout": {
    "columns": 2,
    "widgets": [
      {
        "title": "CPU Instances",
        "xyChart": {
          "dataSets": [
            {
              "timeSeriesQuery": {
                "timeSeriesFilter": {
                  "filter": "metric.type=compute.googleapis.com/instance/cpu/utilization",
                  "aggregation": {
                    "alignmentPeriod": {
                      "seconds": 60,
                      "perSeriesAligner": "ALIGN_MEAN"
                    }
                  }
                }
              }
            }
          ]
        }
      },
      {
        "title": "Disques",
        "xyChart": {
          "dataSets": [
            {
              "timeSeriesQuery": {
                "timeSeriesFilter": {
                  "filter": "metric.type=compute.googleapis.com/instance/disk/read_bytes_count",
                  "aggregation": {
                    "alignmentPeriod": {
                      "seconds": 60,
                      "perSeriesAligner": "ALIGN_RATE"
                    }
                  }
                }
              }
            }
          ]
        }
      }
    ]
  }
}'
```

Antiseche : Ca ne marche pas, je fais quoi ?

Symptome	Cause probable	Action Cloud Shell immediate
L instance ne démarre plus apres changement de taille	Probleme de compatibilite des drivers (ENA, NVMe) ou disque plein	Repasser immediatement a l ancien type d instance. Verifier gcloud compute instances get-serial-port-output.
Le script Python bloque	Timeout IAM ou quota API depasse	export GOOGLE_CLOUD_QUOTA_PROJECT=PROJECT_ID ou verifier les logs Cloud Audit.
Le Cloud Shell rame ou se deconnecte	Session expiree ou surcharge navigateur	Ouvrir un 2eme onglet ou utiliser tmux dans Cloud Shell pour ne pas perdre l historique.
Bucket inaccessible	Permission IAM manquante ou bucket supprime	Verifier gsutil iam get gs://BUCKET et les logs d audit. Activer le soft delete si < 7 jours.
GKE : Pods en CrashLoop-BackOff	Image non trouvee, OOMKilled, ou sonde de sante echouee	kubectl describe pod NOM -> verifier Events. kubectl logs --previous pour le crash precedent.

Table 2 – continued

Symptome	Cause probable	Action Cloud Shell immediate
Cloud SQL inaccessible	IP non autorisee, SSL non force, ou instance en maintenance	Verifier <code>--authorized-networks</code> et <code>--require-ssl</code> . Verifier le statut : <code>gcloud sql instances describe</code> .
Docker : no space left on device	Disque sature par les logs ou images orphelines	<code>docker system prune -a --volumes -f</code> puis <code>docker system df</code> .
Load Balancer renvoie 502	Backend unhealthy ou firewall bloquant les health checks	Verifier <code>gcloud compute backend-services get-health</code> . Autoriser les IPs Google (130.211.0.0/22).
IAM : Permission denied mysterieux	Policy heritee d un folder ou org, ou service account sans role	<code>gcloud iam policies analyze-resource --resource=...</code> pour tracer l origine.
Facture GCP en explosion	Instance ou bucket multi-region non prevu	<code>gcloud billing budgets list</code> puis <code>gcloud recommender recommendations list --recommender=google.compute.instance.IdleResourceRecommender</code> .

Checklist Vendredi Soir

Avant de fermer ton Cloud Shell et de partir en week-end, valide mentalement ces points :

Conteneurs & Compute

- **Les Limits & Requests** : Mon conteneur a-t-il une limite de RAM definie ? (`resources.limits.memory`)
- **La Restart Policy** : `--restart always` pour Docker ? `ReplicaSet > 1` pour K8s ?
- **Les Probes** : Les `livenessProbe` et `readinessProbe` pointent-elles sur le bon port ?
- **Shielded VM** : Secure Boot et Integrity Monitoring actives sur les VMs critiques ?

Securite

- **Firewall temporaire** : Ai-je une regle `allow-admin-temp-*` a supprimer lundi ?
- **Cles de service account** : Ai-je cree des cles temporaires a revoquer ?
- **Buckets publics** : Aucun `allUsers` ou `allAuthenticatedUsers` n a ete ajoute ?
- **Acces SQL** : Les IPs autorisees sont-elles restreintes (pas de 0.0.0.0/0) ?

Backup & Monitoring

- **Snapshots** : Les disques critiques ont-ils un snapshot recent (< 24h) ?
- **Backups SQL** : La derniere backup Cloud SQL est-elle reussie ?
- **Alertes** : Les canaux de notification (email/Slack/PagerDuty) sont-ils actifs ?
- **Logs** : Les Cloud Audit Logs sont-ils exportes vers BigQuery ou Cloud Storage ?

✔ Cout & Gouvernance

- **Instances orphelines** : `gcloud compute instances list --filter="creationTimestamp>=$(date -d '7 days ago' +%Y-%m-%d)"` – tout est justifié ?
- **Buckets inutiles** : `gsutil ls` – chaque bucket a-t-il un lifecycle policy ?
- **Budget alert** : Le budget de billing est-il configuré avec un seuil d'alerte ?

🧹 Nettoyage Post-Crise

Une fois les éléments déployés en prod, planifie la suppression de ce qui est temporaire :

```
# Supprimer les règles de pare-feu temporaires
gcloud compute firewall-rules list --filter="name~'allow-admin-temp'" --format="value(name)" | \
xargs -I {} gcloud compute firewall-rules delete {} --quiet

# Lister les instances créées cette semaine (vérifier les oublis)
gcloud compute instances list \
  --filter="creationTimestamp >= $(date -d '7 days ago' +%Y-%m-%d)" \
  --format="table(name, zone, machineType, creationTimestamp, status)"

# Lister les snapshots vieux de +30 jours (à nettoyer)
gcloud compute snapshots list \
  --filter="creationTimestamp < $(date -d '30 days ago' +%Y-%m-%d)" \
  --format="table(name, diskSizeGb, creationTimestamp)"

# Supprimer les clés de service account temporaires
gcloud iam service-accounts keys list \
  --iam-account=SA@PROJECT.iam.gserviceaccount.com \
  --managed-by=user --format="value(name)" | \
xargs -I {} gcloud iam service-accounts keys delete {} \
  --iam-account=SA@PROJECT.iam.gserviceaccount.com

# Vérifier les buckets sans lifecycle policy
for bucket in $(gsutil ls); do
  gsutil lifecycle get "$bucket" 2>/dev/null | grep -q "No lifecycle" && echo
  ↪ "$${vcenter}{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
  ↪ 26a0-fe0f.png}}}}}$ $ Pas de lifecycle: $bucket"
done
```

📖 References & Ressources

Ressource	Lien	Usage
Cloud Shell	console.cloud.google.com	Terminal de secours dans le navigateur
Cloud Status	status.cloud.google.com	Vérifier une panne régionale
Pricing Calculator	cloud.google.com/products/calculator	Estimer le coût avant de créer

Table 3 – continued

Ressource	Lien	Usage
IAM Recommender	<code>gcloud recommender recommendations list</code>	Optimiser les permissions
Security Command Center	<code>gcloud scc assets list</code>	Vue d ensemble de la securite
Cloud Logging	<code>gcloud logging read</code>	Forensics et audit
Documentation officielle	cloud.google.com/docs	La source de verite

 **Note finale** : Ce guide est vivant. Fork-le, adapte-le a ton organisation, et teste chaque commande en **staging** avant de l utiliser en production. Le vendredi a 16h55 n est pas le moment d apprendre une nouvelle commande.

“En crise, on ne monte pas au niveau de ses attentes, on descend au niveau de sa preparation.” –
Adapte de Archilochus